

Report for AI Capstone Homework 4: A Robot Manipulation Framework

FU, YONG-WEI (112550107)

May 10, 2026

1 Task 1: Forward Kinematics

1.1 Implementation of ‘your_fk()’ function

The implementation of my `your_fk()` function follows the standard Denavit-Hartenberg (D-H) procedure:

1. **Transformation Chain:** For each joint $i \in \{1, \dots, 6\}$, a 4×4 transformation matrix A_i is constructed using the given D-H parameters (a_i, α_i, d_i) and the input joint angle θ_i . The matrix follows the D-H convention:

$$A_i = \begin{bmatrix} \cos \theta_i & -\sin \theta_i \cos \alpha_i & \sin \theta_i \sin \alpha_i & a_i \cos \theta_i \\ \sin \theta_i & \cos \theta_i \cos \alpha_i & -\cos \theta_i \sin \alpha_i & a_i \sin \theta_i \\ 0 & \sin \alpha_i & \cos \alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

```
# T_list stores the transformation matrix from the base to each joint frame.  
# T_list[0] is the base transformation matrix.  
T_list = [A]
```

Figure 1: Code snippet for constructing the D-H transformation matrix.

2. **Cumulative Multiplication:** The matrices are multiplied starting from the base frame transformation (A_{base}) to compute the transformation from the world frame to the end-effector using: $T = A_{base} \cdot A_1 \cdot A_2 \cdots A_6$.

```

for i in range(len(q)):
    theta = q[i]
    a = DH_params[i]['a']
    d = DH_params[i]['d']
    alpha = DH_params[i]['alpha']

    # Classic DH transformation matrix from frame i-1 to frame i
    ct = np.cos(theta)
    st = np.sin(theta)
    ca = np.cos(alpha)
    sa = np.sin(alpha)

    T_i = np.array([
        [ct, -st * ca, st * sa, a * ct],
        [st, ct * ca, -ct * sa, a * st],
        [0, sa, ca, d],
        [0, 0, 0, 1]
    ], dtype=np.float64)

    # Cumulative transformation from base to current joint frame
    A = A @ T_i
    T_list.append(A.copy())

    # End-effector position in world frame
    p_end = A[:3, 3]

```

Figure 2: Code snippet for cumulative multiplication of transformation matrices.

3. **Geometric Jacobian Construction:** The 6×6 Jacobian matrix J is built column by column. For each revolute joint i , the i -th column is defined as:

$$J_i = \begin{bmatrix} z_{i-1} \times (p_{end} - p_{i-1}) \\ z_{i-1} \end{bmatrix}$$

where z_{i-1} is the z -axis of frame $i - 1$ and p_{i-1} is its origin, both expressed in the base frame.

```

for i in range(6):
    z_prev = T_list[i][:3, 2]
    p_prev = T_list[i][:3, 3]

    # Linear velocity component (Jv)
    jacobian[:3, i] = np.cross(z_prev, p_end - p_prev)
    # Angular velocity component (Jw)
    jacobian[3:, i] = z_prev

```

Figure 3: Code snippet for constructing the geometric Jacobian.

1.2 What is the difference between D-H convention and Craig's convention (Modified D-H Convention)?

The primary differences lie in the frame placement and the order of operations:

- **Frame Assignment:** In the **Classic D-H convention**, frame i is attached to the end of link i , and the joint i axis is z_{i-1} . In **Craig's (Modified) D-H convention**, frame i is attached to the beginning of link i , and the joint i axis is z_i .

- **Transformation Order:**

- **Classic:** $A_i = Rot_z(\theta_i) \cdot Trans_z(d_i) \cdot Trans_x(a_i) \cdot Rot_x(\alpha_i)$. The parameters relate frame $i - 1$ to frame i along the common normal and the joint axis.
- **Modified:** $A_i = Rot_x(\alpha_{i-1}) \cdot Trans_x(a_{i-1}) \cdot Rot_z(\theta_i) \cdot Trans_z(d_i)$. The parameters relate frame i to frame $i - 1$ (with indices shifted).

1.3 Complete the D-H table

The parameters used in the implementation (based on the UR5 assignment spec) are as follows:

i	d (m)	α (rad)	a (m)	θ (rad)
1	0.0892	$\pi/2$	0	θ_1
2	0	0	-0.425	θ_2
3	0	0	-0.392	θ_3
4	0.1093	$\pi/2$	0	θ_4
5	0.09475	$-\pi/2$	0	θ_5
6	0.2023	0	0	θ_6
7	0	0	0	0 (Fixed)

Table 1: Classic D-H parameters for the UR5 robot configuration (including fixed tool frame $i = 7$).

2 Task 2: Inverse Kinematics

2.1 Implementation of ‘your_ik()’ function

My `your_ik()` function implements a **Numerical Inverse Kinematics** solver using the Jacobian Pseudo-inverse method:

1. **Initialization:** The joints are initialized with a given guess `q_init`.
2. **Iteration Loop:** In each iteration (up to `max_iters`):
 - Calculate the current end-effector pose and Jacobian using `your_fk()` I implemented earlier.
 - **Error Calculation:** Compute the 6D error vector Δx . The position error is $p_{target} - p_{curr}$. The orientation error is derived by converting the relative rotation $R_{target} \cdot R_{curr}^T$ into its axis-angle representation.
 - **Joint Update:** Compute the joint update $\Delta q = \alpha \cdot J^\dagger \cdot \Delta x$, where $J^\dagger$ is the Moore-Penrose pseudo-inverse and α is the step size (learning rate).
 - **Constraints:** Clip the updated joint angles q within the hardware `joint_limits`.
3. **Termination:** The loop stops if the norm of the 6D error falls below `stop_thresh`.

```

for i in range(max_iters):
    # 1. Get current pose and Jacobian
    curr_pose, jacobian = your_fk(dh_params, tmp_q, base_pos)
    curr_pos = curr_pose[:3]
    curr_quat = curr_pose[3:]
    R_curr = R.from_quat(curr_quat).as_matrix()

    # 2. Compute position error
    pos_error = target_pos - curr_pos

    # 3. Compute orientation error (axis-angle of R_target @ R_curr.T)
    R_error = R_target @ R_curr.T
    rot_error_vec = R.from_matrix(R_error).as_rotvec()

    # 4. Combine into 6D error
    error_6d = np.concatenate([pos_error, rot_error_vec])

    # 5. Check termination
    if np.linalg.norm(error_6d) < stop_thresh:
        break

    # 6. Compute delta_q = pinv(J) @ error
    # Use a small dampening factor for stability near singularities
    # delta_q = pinv(jacobian) @ error_6d
    # Or more simply with pinv from scipy:
    delta_q = pinv(jacobian) @ error_6d

    # 7. Update tmp_q and clip
    tmp_q += step_size * delta_q

    # Joint limits clipping
    for j in range(6):
        tmp_q[j] = np.clip(tmp_q[j], joint_limits[j, 0], joint_limits[j, 1])

```

Figure 4: Code snippet for the iterative IK solver using the Jacobian pseudo-inverse method.

2.2 What problems do you encounter and how do you deal with them?

The most significant problem encountered was that neither the headless nor the GUI tests for Task 1 and Task 2 could be executed on the glows.ai environment. The simulation would fail to initialize the graphics or physics engine, such that only a simulation window with empty content would pop up, and then fail with segmentation faults.

To resolve this, I modified the `SimulationApp` initialization logic in the scripts. Specifically, I explicitly configured the environment and GPU settings as follows:

- **Environment Variables:** Manually set `CUDA_VISIBLE_DEVICES` and `NVIDIA_VISIBLE_DEVICES` to “0” to force the application to recognize the GPU as a single-device.
- **Licensing:** Set `OMNI_KIT_ACCEPT_EULA` and `PRIVACY_CONSENT` to “Y” to bypass interactive prompts in a CLI/Headless environment.
- **GPU Configuration:** Replaced the default `SimulationApp` initialization with a detailed dictionary that explicitly assigned `active_gpu` and `physics_gpu` to index 0 and disabled `multi_gpu` support.

This modification was essential for the simulator to successfully bypass hardware detection errors and carry out the kinematics calculations for the test cases.